Lesson 4: Iterated Filtering — Principles and Practice

Aaron A. King Edward L. Ionides Kunyang He

Saturday, March 21, 2026

Objectives

This lesson covers likelihood maximization via the method of iterated filtering (IF2). It presupposes familiarity with building POMP models and particle filtering from Lessons 2 and 3.

1. Review the available options for inference on POMP models, to put iterated filtering in context.
2. Understand how iterated filtering algorithms carry out repeated particle filtering operations, with randomly perturbed parameter values, in order to maximize the likelihood.
3. Gain experience carrying out statistical investigations using iterated filtering in a relatively simple situation: fitting an SIR model to data from a measles outbreak.

The plug-and-play property

Inference methodology that calls `rproc` but not `dprocess` is said to be **plug-and-play**. All popular modern Monte Carlo methods for POMP models are in this category. "Simulation-based" is equivalent to "plug-and-play".

- Plug-and-play methods can call `dmeas`. A method that uses only `rproc` and `rmeas` is called "doubly plug-and-play".
- Two non-plug-and-play methods — **EM** and **MCMC** — have theoretical convergence problems for nonlinear POMP models.

POMP inference methodologies

	Frequentist	Bayesian
Plug-and-play, Full-info	Iterated filtering	Particle MCMC
Plug-and-play, Feature	Simulated moments	ABC
Not plug-and-play, Full-info	EM, Kalman filter	MCMC

Full-information, plug-and-play, frequentist methods

Iterated filtering methods are the only currently available, full-information, plug-and-play, frequentist methods for POMP models.

In the **IF2 algorithm**:

1. Each iteration consists of a particle filter, carried out with the parameter vector, for each particle, doing a random walk.

2. At the end of the time series, the collection of parameter vectors is recycled as starting parameters for the next iteration.

3. The random-walk variance decreases at each iteration.

Analogy with natural selection

- The **parameters** characterize the genotype
- The **swarm of particles** is a population
- The **likelihood** is the analogue of **fitness**
- Each successive observation is a **new generation**

- Since particles reproduce in proportion to their likelihood, the particle filter acts like **natural selection**
- The artificial perturbations correspond to **mutation**
- IF2 increases the fitness of the population of particles

Setting up the model

```
import jax.numpy as jnp
import jax
import pandas as pd
import numpy as np
import pypomp as pp
import matplotlib.pyplot as plt
import pickle
import os
import time
from scipy.stats import chi2

cache_dir = "cache"
os.makedirs(cache_dir, exist_ok=True)
```

```
file = "Measles_Consett_1948.csv"
meas = (
    pd.read_csv(file)
    .loc[:, ["week", "cases"]]
    .rename(columns={"week": "time",
                     "cases": "reports"})
    .set_index("time")
    .astype(float)
)

ys = meas.copy()
ys.columns = pd.Index(["reports"])
```

Negative binomial functions

- These allow for overdispersed measurement distributions

```
def nbinom_logpmf(x, k, mu):
    x, k, mu = (jnp.asarray(v) for v in (x, k, mu))
    logp_zero = jnp.where(x == 0, 0.0, -jnp.inf)
    safe_mu = jnp.where(mu == 0.0, 1.0, mu)
    gammaln = jax.scipy.special.gammaln
    core = (gammaln(k + x) - gammaln(k)
            - gammaln(x + 1)
            + k * jnp.log(k / (k + safe_mu))
            + x * jnp.log(safe_mu / (k + safe_mu)))
    return jnp.where(mu == 0.0, logp_zero, core)

def rnbinom(key, k, mu):
    key_g, key_p = jax.random.split(key)
    lam = jax.random.gamma(key_g, k) * (mu / k)
    return jax.random.poisson(key_p, lam)
```

SIR model components

```
def rinit(theta_, key, covars, t0):
    N = theta_["N"]
    eta = theta_["eta"]
    S0 = jnp.round(N * eta)
    I0 = 1.0
    R0 = jnp.round(N * (1 - eta)) - 1.0
    H0 = 0.0
    return {"S": S0, "I": I0, "R": R0, "H": H0}
```

```
def rproc(X_, theta_, key, covars, t, dt):
    S = jnp.asarray(X_["S"])
    I = jnp.asarray(X_["I"])
    R = jnp.asarray(X_["R"])
    H = jnp.asarray(X_["H"])
    Beta = theta_["Beta"]
    mu_IR = theta_["mu_IR"]
    N = theta_["N"]

    p_SI = 1.0 - jnp.exp(-Beta * I / N * dt)
    p_IR = 1.0 - jnp.exp(-mu_IR * dt)

    key_SI, key_IR = jax.random.split(key)
    dN_SI = jax.random.binomial(
        key_SI, n=jnp.int32(S), p=p_SI)
    dN_IR = jax.random.binomial(
        key_IR, n=jnp.int32(I), p=p_IR)

    return {"S": S - dN_SI,
            "I": I + dN_SI - dN_IR,
            "R": R + dN_IR,
            "H": H + dN_IR}
```

```
def dmeas(Y_, X_, theta_, covars, t):
    rho = theta_["rho"]
    k = theta_["k"]
    H = X_["H"]
```

```
    mu = rho * H
    return nbinom_logpmf(Y_["reports"], k, mu)

def rmeas(X_, theta_, key, covars, t):
    rho = theta_["rho"]
    k = theta_["k"]
    H = X_["H"]
    mu = rho * H
    reports = rnbinom(key, k, mu)
    return jnp.array([reports])
```

Parameter transformations

In the R version, `partrans=parameter_trans(log="Beta", logit=c("rho","eta"))` ensures IF2 random walks respect parameter constraints. In `pypomp`, we use `ParTrans` to achieve the same. The parameter names remain unchanged; only the values are transformed internally.

```
def to_est(theta):
    """Natural scale -> estimation scale."""
    return {
        "Beta": jnp.log(theta["Beta"]),
        "mu_IR": theta["mu_IR"],
        "N": theta["N"],
        "rho": jax.scipy.special.logit(theta["rho"]),
        "eta": jax.scipy.special.logit(theta["eta"]),
        "k": theta["k"],
    }

def from_est(theta):
    """Estimation scale -> natural scale."""
    return {
        "Beta": jnp.exp(theta["Beta"]),
        "mu_IR": theta["mu_IR"],
        "N": theta["N"],
        "rho": jax.scipy.special.expit(theta["rho"]),
        "eta": jax.scipy.special.expit(theta["eta"]),
        "k": theta["k"],
    }

par_trans = pp.ParTrans(
    to_est=to_est, from_est=from_est)
```

Create POMP object

```
theta = {
    "Beta": 15.0,
    "mu_IR": 2.0,
    "N": 38000.0,
    "eta": 0.06,
    "rho": 0.5,
    "k": 10.0
}

fixed_params = {"N": 38000.0, "mu_IR": 2.0, "k": 10.0}
statenames = ["S", "I", "R", "H"]

measSIR = pp.Pomp(
    rinit=rinit, rproc=rproc,
    dmeas=dmeas, rmeas=rmeas,
    ys=ys, theta=theta,
    statenames=statenames,
    par_trans=par_trans,
    t0=0.0, nstep=7,
    accumvars=("H",),
```

```
    ydim=1, covars=None
)
```

Specifying random walk standard deviations

The random walk perturbations are applied on the **estimation** (transformed) scale. A perturbation of 0.02 on the log scale corresponds to a ~2% multiplicative change; on the logit scale, it gives a small additive shift in log-odds.

```
rw_sd = pp.RWSigma(
    sigmas={
        "Beta": 0.02,
        "mu_IR": 0.0,
        "N": 0.0,
        "rho": 0.02,
        "eta": 0.02,
        "k": 0.0
    },
    init_names=["eta"]
)
```

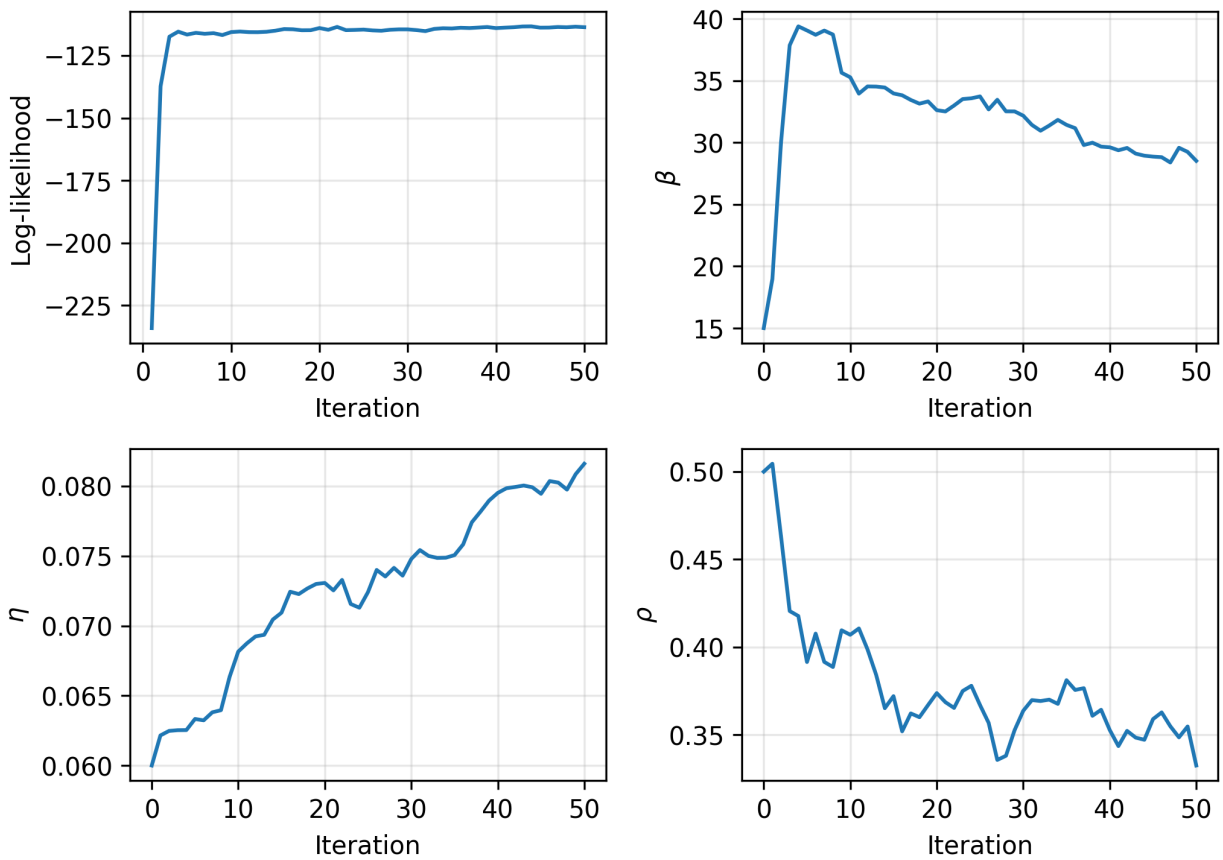
This is analogous to `rw.sd=rw_sd(Beta=0.02, rho=0.02, eta=ivp(0.02))` with `partrans=parameter_trans(lo` `logit=c("rho","eta"))` in R-pomp.

Local search

```
cache_file = cache_dir + "/local-mif.pkl"

if os.path.exists(cache_file):
    with open(cache_file, 'rb') as f:
        mif_result = pickle.load(f)
else:
    key = jax.random.key(482947940)
    measSIR.mif(
        J=2000, M=50,
        rw_sd=rw_sd, a=0.5, key=key
    )
    mif_result = measSIR.results_history.last()
    with open(cache_file, 'wb') as f:
        pickle.dump(mif_result, f)
```

```
traces_da = mif_result.traces_da
traces_rep0 = traces_da.isel(replicate=0)
iterations = traces_rep0.coords['iteration'].values
variables = traces_rep0.coords['variable'].values

fig, axes = plt.subplots(2, 2, figsize=(7, 5))

for ax, var, lab in zip(
    axes.flat,
    ['logLik', 'Beta', 'eta', 'rho'],
    ['Log-likelihood', r'$\beta$',
     r'$\eta$', r'$\rho$']
):
    if var in variables:
        ax.plot(iterations,
            traces_rep0.sel(variable=var).values)
    ax.set(xlabel='Iteration', ylabel=lab)
    ax.grid(alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```



Estimating the likelihood

The IF2 log-likelihood is not a reliable estimate. We evaluate the likelihood at the IF2 endpoint using `pfilter`:

```
cache_file = cache_dir + "/local-pfilter.pkl"

if os.path.exists(cache_file):
    with open(cache_file, 'rb') as f:
        pf_result = pickle.load(f)
else:
    key = jax.random.key(900242057)
    measSIR.pfilter(key=key, J=5000, reps=10)
    pf_result = measSIR.results_history.last()
    with open(cache_file, 'wb') as f:
        pickle.dump(pf_result, f)

logliks = pf_result.logLiks.values.flatten()
ll_est = pp.logmeanexp(logliks)
ll_se = pp.logmeanexp_se(logliks)
print(f"Log-likelihood: {ll_est:.1f}",
      f"(SE: {ll_se:.1f})")
```

```
Log-likelihood: -112.5 (SE: 0.0)
```

A global search of the likelihood surface

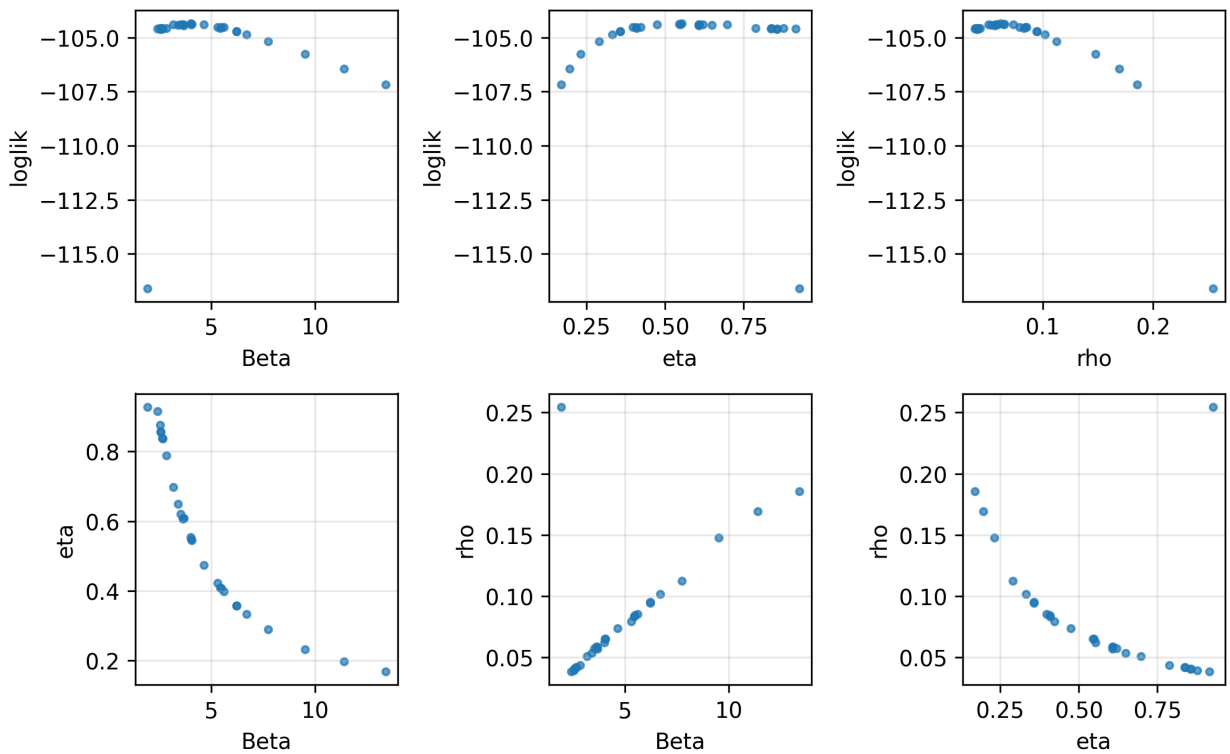
We take advantage of `pypomp`'s native support for multiple parameter sets: passing a list of dicts to `Pomp` causes `mif` and `pfilter` to run all replicates **in parallel on the GPU**.

```
cache_file = cache_dir + "/global-search.pkl"

if os.path.exists(cache_file):
    with open(cache_file, 'rb') as f:
        results_df = pickle.load(f)
else:
    np.random.seed(2062379496)
    n_starts = 40
    theta_list = []
    for i in range(n_starts):
        th = theta.copy()
        th["Beta"] = np.random.uniform(5, 80)
        th["rho"] = np.random.uniform(0.2, 0.9)
        th["eta"] = np.random.uniform(0.01, 0.99)
        theta_list.append(th)

    # All starts run in parallel on GPU
    mod = pp.Pomp(
        rinit=rinit, rproc=rproc,
        dmeas=dmeas, rmeas=rmeas,
        ys=ys, theta=theta_list,
        statenames=statenames,
        par_trans=par_trans,
        t0=0.0, nstep=7,
        accumvars=("H",),
        ydim=1, covars=None
    )
    key = jax.random.key(1270401374)
    mod.mif(J=2000, M=50,
            rw_sd=rw_sd, a=0.5, key=key)
    # Continue from endpoints with 100 more
    key = jax.random.key(1270401375)
    mod.mif(J=2000, M=100,
            rw_sd=rw_sd, a=0.5, key=key)

    # Evaluate likelihoods in parallel
    key = jax.random.key(1270401376)
    mod.pfilter(key=key, J=5000, reps=10)
    pf = mod.results_history.last()

    rows = []
    for i in range(n_starts):
        lls = pf.logLiks.values[i, :]
        rows.append({
            **mod.theta[i],
            'loglik': pp.logmeanexp(lls),
            'loglik_se': pp.logmeanexp_se(lls)
        })
    results_df = pd.DataFrame(rows)
    results_df = results_df[
        np.isfinite(results_df['loglik'])]
    with open(cache_file, 'wb') as f:
        pickle.dump(results_df, f)

print(f"Best log-likelihood:",
      f"{results_df['loglik'].max():.1f}")
```

```
Best log-likelihood: -104.4
```

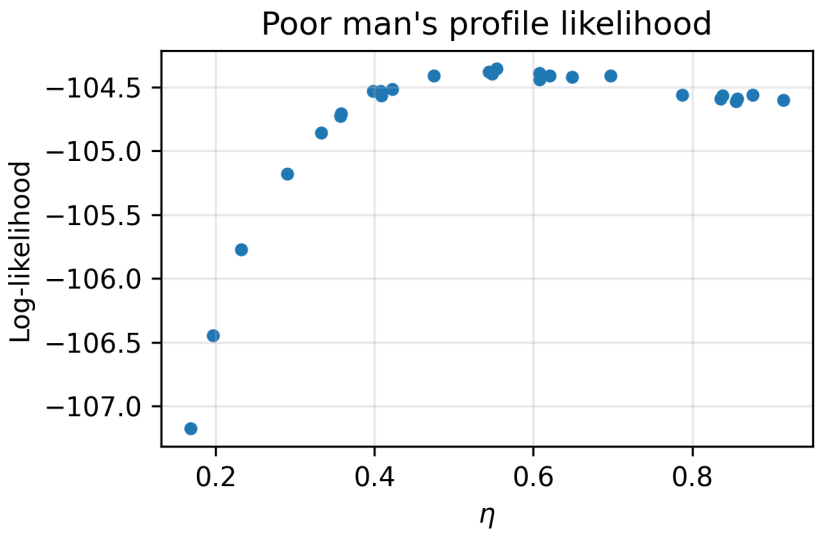
Visualizing the global search

```
fig, axes = plt.subplots(2, 3, figsize=(8, 5))
pairs = [('Beta','loglik'), ('eta','loglik'),
         ('rho','loglik'), ('Beta','eta'),
         ('Beta','rho'), ('eta','rho')]
for ax, (x, y) in zip(axes.flat, pairs):
    ax.scatter(results_df[x], results_df[y],
        s=10, alpha=0.7)
    ax.set(xlabel=x, ylabel=y)
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



The estimates converge on a particular region in parameter space despite starting from diverse remote starting points. The projections give us "poor man's profiles":

```
fig, ax = plt.subplots(figsize=(4.5, 3))
top = results_df[
    results_df['loglik'] > results_df['loglik'].max() - 10]
ax.scatter(top['eta'], top['loglik'], s=15)
ax.set(xlabel=r'$\eta$', ylabel='Log-likelihood',
    title="Poor man's profile likelihood")
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



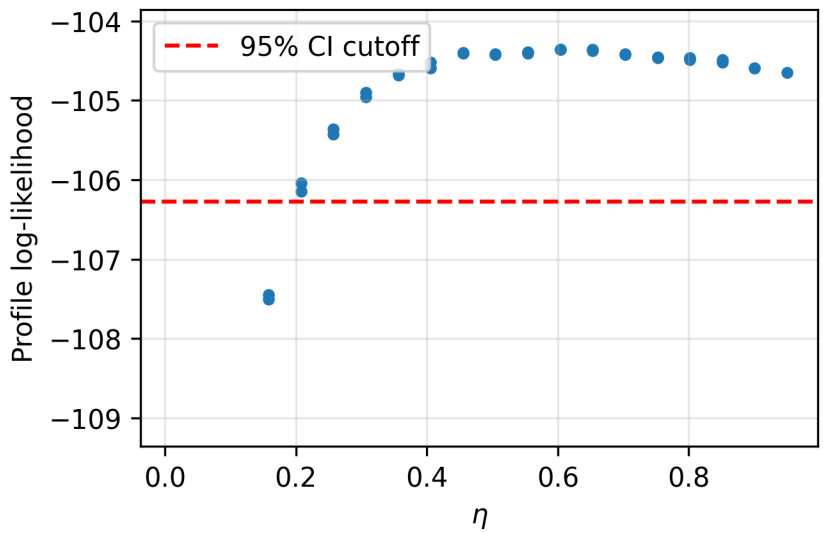
Computing a profile over η

To profile over η, we fix η at a grid of values and optimize over the remaining parameters. For each grid point, we launch multiple starts **in parallel**.

```
cache_file = cache_dir + "/profile-eta.pkl"

rw_sd_profile = pp.RWSigma(
    sigmas={
        "Beta": 0.02, "mu_IR": 0.0,
        "N": 0.0, "rho": 0.02,
        "eta": 0.0, "k": 0.0
    },
    init_names=[]
)

if os.path.exists(cache_file):
    with open(cache_file, 'rb') as f:
        profile_eta_df = pickle.load(f)
else:
    np.random.seed(1196696958)
    eta_grid = np.linspace(0.01, 0.95, 20)
    n_prof = 5
    profile_eta = []

    for i, eta_val in enumerate(eta_grid):
        # Build n_prof starts for this grid point
        theta_list = []
        for j in range(n_prof):
            th = theta.copy()
            th["eta"] = float(eta_val)
            th["Beta"] = np.random.uniform(5, 40)
            th["rho"] = np.random.uniform(0.2, 0.9)
            theta_list.append(th)

        mod = pp.Pomp(
            rinit=rinit, rproc=rproc,
            dmeas=dmeas, rmeas=rmeas,
            ys=ys, theta=theta_list,
            statenames=statenames,
            par_trans=par_trans,
            t0=0.0, nstep=7,
            accumvars=("H",),
            ydim=1, covars=None
```

```
        )
        key = jax.random.key(i * 1000)
        mod.mif(J=2000, M=100,
            rw_sd=rw_sd_profile, a=0.3, key=key)
        key = jax.random.key(i * 1000 + 3000)
        mod.mif(J=2000, M=100,
            rw_sd=rw_sd_profile, a=0.3, key=key)

        key = jax.random.key(i * 1000 + 5000)
        mod.pfilter(key=key, J=5000, reps=10)
        pf = mod.results_history.last()

        for j in range(n_prof):
            lls = pf.logLiks.values[j, :]
            fp = mod.theta[j]
            profile_eta.append({
                'eta': eta_val,
                'Beta': fp['Beta'],
                'rho': fp['rho'],
                'loglik': pp.logmeanexp(lls),
                'loglik_se': pp.logmeanexp_se(lls)
            })

    profile_eta_df = pd.DataFrame(profile_eta)
    profile_eta_df = profile_eta_df[
        np.isfinite(profile_eta_df['loglik'])]
    with open(cache_file, 'wb') as f:
        pickle.dump(profile_eta_df, f)
```

```
max_ll = profile_eta_df['loglik'].max()
ci_cutoff = max_ll - 0.5 * chi2.ppf(0.95, df=1)

# Keep top 2 per grid point
top = (profile_eta_df
    .groupby(profile_eta_df['eta'].round(5))
    .apply(lambda g: g.nlargest(2, 'loglik'),
        include_groups=False)
    .reset_index(drop=True))

fig, ax = plt.subplots(figsize=(4.5, 3))
ax.scatter(top['eta'], top['loglik'], s=12)
ax.axhline(ci_cutoff, color='red',
    linestyle='--', label='95% CI cutoff')
ax.set(xlabel=r'$\eta$',
    ylabel='Profile log-likelihood')
ax.set_ylim(max_ll - 5, max_ll + 0.5)
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



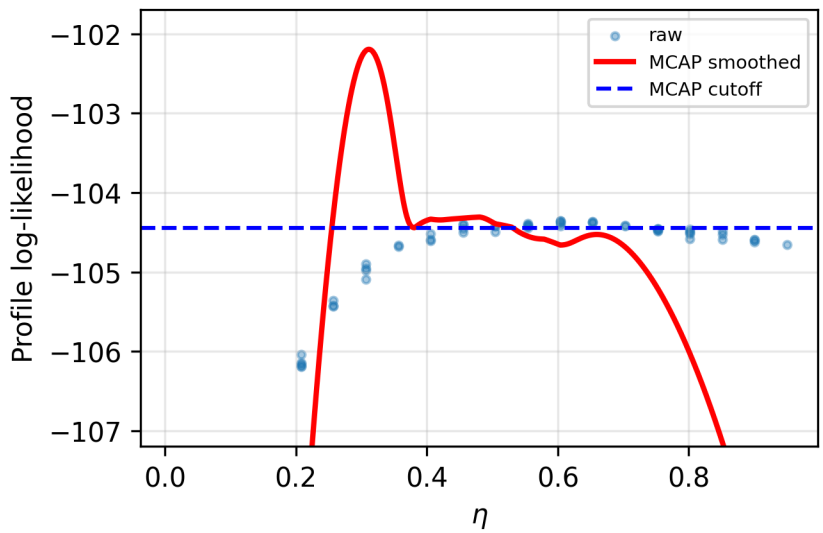
Monte Carlo Adjusted Profile (MCAP)

The `mcap` function in `pypomp` constructs a profile likelihood confidence interval that accounts for Monte Carlo error in the profile:

```
mcap_result = pp.mcap(
    parameter=profile_eta_df['eta'].values,
    loglik=profile_eta_df['loglik'].values,
    level=0.95, span=0.75)

print(f"MLE of eta: {mcap_result.mle:.4f}")
print(f"95% CI: ({mcap_result.ci[0]:.4f},"
      f" {mcap_result.ci[1]:.4f})")
```

```
MLE of eta: 0.3111
95% CI: (0.2556, 0.5303)
```

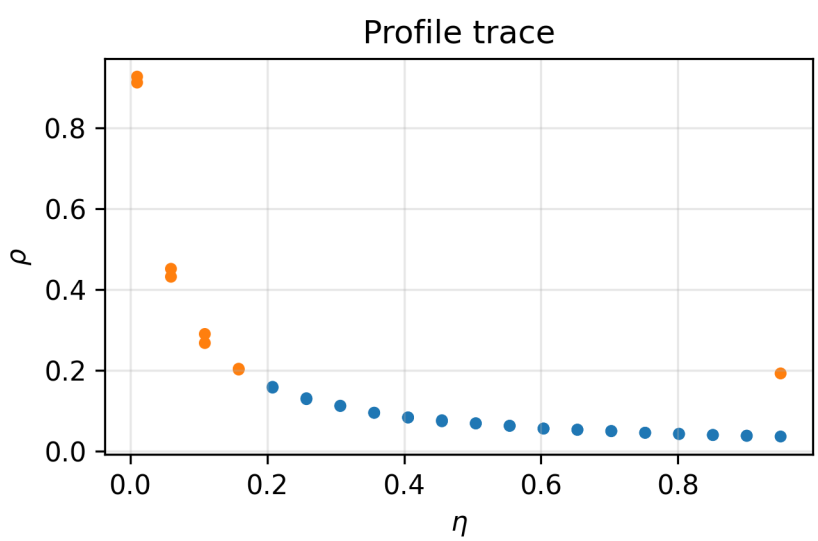
```
fig, ax = plt.subplots(figsize=(4.5, 3))
ax.scatter(profile_eta_df['eta'],
    profile_eta_df['loglik'],
    s=8, alpha=0.4, label='raw')
ax.plot(mcap_result.fit['parameter'],
    mcap_result.fit['smoothed'],
    'r-', lw=2, label='MCAP smoothed')
max_sm = np.nanmax(mcap_result.fit['smoothed'])
ax.axhline(max_sm - mcap_result.delta,
    color='blue', linestyle='--',
    label='MCAP cutoff')
ax.set(xlabel=r'$\eta$',
    ylabel='Profile log-likelihood')
ax.set_ylim(max_sm - 5, max_sm + 0.5)
ax.legend(fontsize=7)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Profile trace

As one varies η across the profile, the model compensates by adjusting the other parameters. A **profile trace** shows how ρ changes as η is varied:

```
top['in_ci'] = top['loglik'] > max_ll - 1.92

fig, ax = plt.subplots(figsize=(4.5, 3))
colors = top['in_ci'].map(
    {True: 'C0', False: 'C1'})
ax.scatter(top['eta'], top['rho'],
    c=colors, s=12)
ax.set(xlabel=r'$\eta$', ylabel=r'$\rho$',
    title='Profile trace')
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Exercises

Exercise 4.1: Estimate SEIR model parameters using IF2. Track computation time and use run-levels.

Exercise 4.2: Estimate all SIR parameters including k. How does the fit improve?

Exercise 4.3: Construct profile likelihood over β and $\mathcal{R}_0$. Which is better determined?

Exercise 4.4: Does your code implement the described model? What are solutions for reproducibility?

Exercise 4.5: How would you identify and fix errors in `rproc`?

Summary

1. IF2 is the primary full-information, plug-and-play, frequentist method for POMP models.

2. The algorithm perturbs parameters with decreasing random walks and resamples both states and parameters.

3. Parameter transformations via `ParTrans(to_est, from_est)` ensure the random walk respects constraints (e.g., `log` for positive parameters, `logit` for probabilities). This is analogous to `partrans=parameter_trans(log="Beta", logit=c("rho","eta"))` in R-pomp.

4. `pypomp` API for IF2: `mif(J, M, rw_sd, a, key)` runs IF2, `RWSigma(sigmas={...}, init_names=[...])` specifies perturbations, `results_history.last().traces_da` gets parameter traces.

5. Passing a **list of parameter dicts** to `Pomp` runs all replicates in parallel on the GPU — use this for global searches and profiles.

6. Always evaluate likelihood at IF2 endpoints with `pfilter`. Use `mcap` for Monte Carlo-adjusted profile confidence intervals.

References

References